Experiment 4

Student Name: Deepanshu Mandhyan **UID: 23BCS12327**
Branch: CSE **Section/Group: KRG 3-A**
Semester: 6th **Date of Performance:06/02/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. Aim:

To design and analyze a scalable OTT (Over-The-Top) video streaming platform similar to Netflix / Amazon Prime, which allows users to register, subscribe, search, and stream video content efficiently while ensuring high availability, low latency, and massive scalability using modern distributed system concepts

2. Objective:

• To design a scalable OTT video streaming platform for efficient content delivery.
• To provide secure user registration and subscription management.
• To enable fast search and seamless video streaming with low latency.
• To ensure high availability and fault tolerance using distributed systems.
• To support massive scalability and performance optimization for large user bases.

3. Tools Used:

- **Draw.io** – System Architecture & HLD Diagram
- **MySQL** – User, subscription, and payment data
- **MongoDB / NoSQL DB** – Video metadata storage
- **ElasticSearch** – Search functionality for movies and TV shows
- **Apache Kafka** – Asynchronous event streaming and video pipeline coordination
- **Redis / CDN Cache** – Caching frequently accessed content
- **Blob Storage (Amazon S3 equivalent)** – Video and media storage

4. System Requirements:

A. Functional Requirements

- The user should be able to create an account on the OTT platform.
- The user should be able to log in and choose an appropriate subscription plan.
- The user should be able to search movies and TV shows using titles or names (Elastic Search).
- The user should be able to stream movies and TV shows in multiple resolutions (480p, 720p, 1080p, 4K).
- The system should provide personalized recommendations for movies and TV shows

B. Non – Functional Requirements

- **Scalability**
  200-300M, for which let's say total videos we are having are 20K videos (~1 hour each)
- **Availability and Consistency**

CAP Theorem: Availability >>>>> Consistency
Availability on watching TV shows and movies
Consistency in making payments and in subscription plans

- **Latency:** 50 - 80 ms

Client should be able to see the video with zero or neglibile buffering.

7. High-Level Design (HLD):

The system follows a MICRO-SERVICE ARCHITECTURE / DISTRIBUTED:

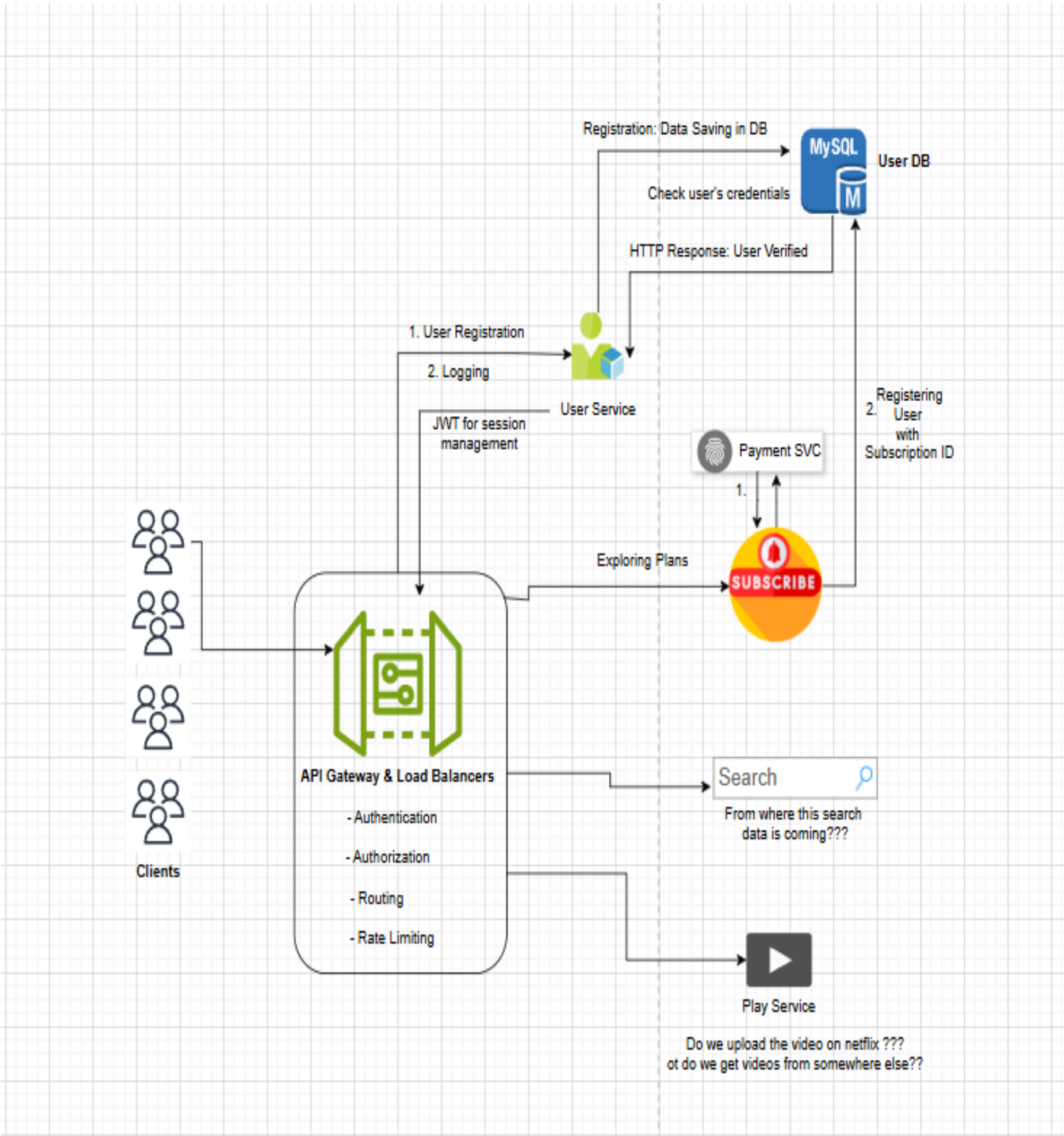

Fig. (b): HLD Diagram

8. Low- Level Design (LLD):

23BCS12327

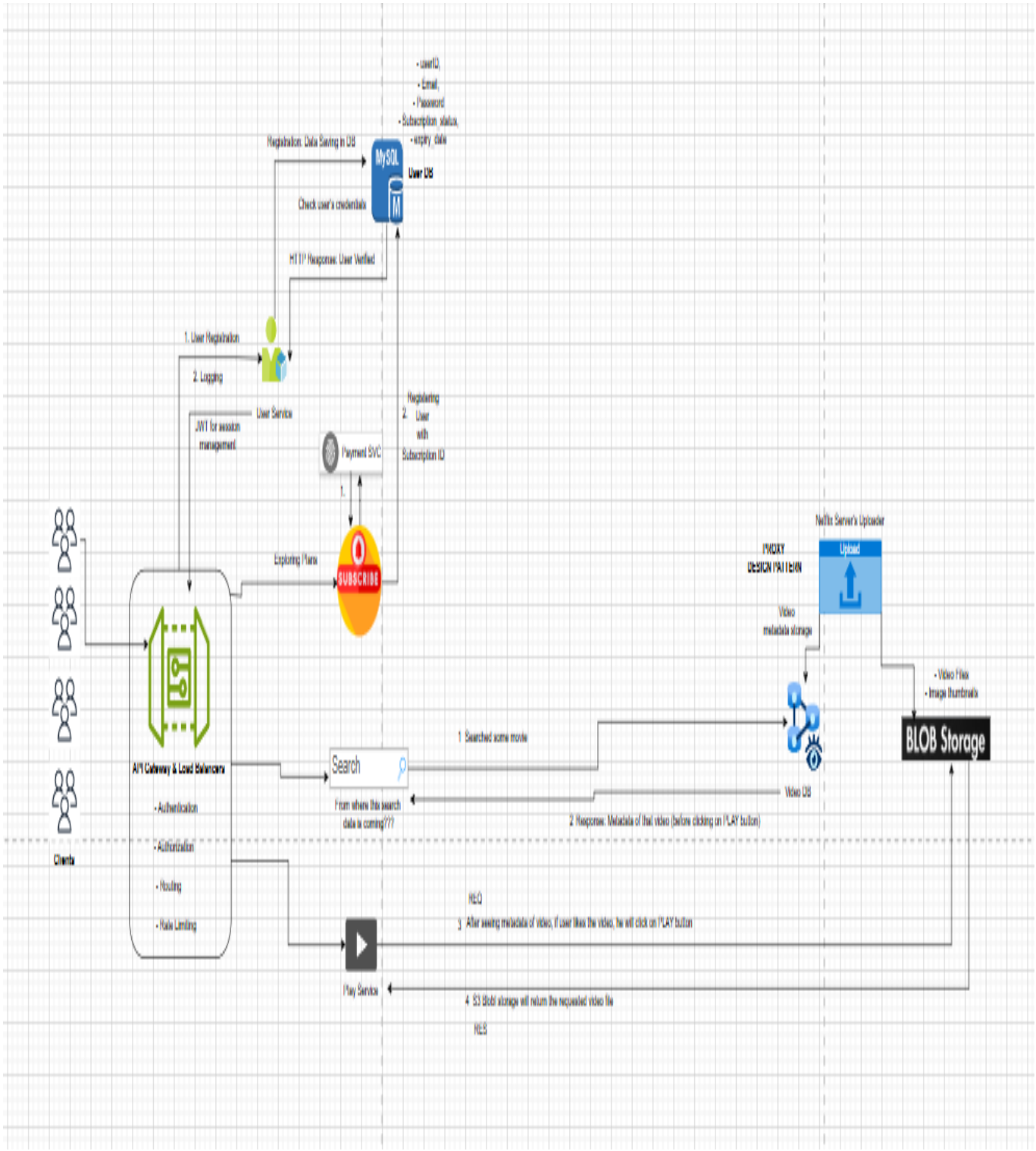

Fig. (c): LLD Diagram

9. Scalability Solution:

- Horizontal scaling is used to add multiple service instances to handle increasing user traffic.
- Load balancers are implemented to distribute requests evenly across backend servers.
- Content Delivery Networks (CDNs) are used to cache videos closer to users for faster streaming.
- Microservices architecture enables independent scaling of critical services like streaming and search.
- Auto-scaling mechanisms dynamically adjust resources based on real-time demand.

10. Learning Outcomes:

- Understand the architecture of a large-scale OTT video streaming platform.
- Learn how distributed systems ensure scalability, availability, and fault tolerance.
- Gain knowledge of adaptive video streaming and CDN-based content delivery.
- Analyze real-world trade-offs using CAP theorem in system design.
- Develop skills to design and evaluate scalable, high-performance web applications.